

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Artificial Intelligence and Data Science
ASSESSMENT TOOL 3 – Comparative Analysis

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO4 – Precision	Assessment:	Assessment Tool 3 – Comparative Analysis
Weightage:	30% of CIA		
CO5 Statement:	Formulate context-free grammars, feature structures, probabilistic parsing techniques and to construct parsing frameworks. (BL-4)		
Student Name:	Mohammed Rayan Shaikh	Reg. No.:	192424059
Section / Batch:	Slot A	Date:	14 - 08 - 2026

Code of Conduct

*I **Mohammed Rayan Shaikh - 192424059** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: _____

Instructions

- Read each scenario carefully before answering.
- Identify the NLP techniques being compared in the question.
- Analyze each approach based on its working principles, advantages, limitations, and applicability.
- Compare the alternatives using technical reasoning rather than listing definitions.
- Support your analysis with examples from the given scenario.
- Duration: 30 minutes.

Section / Topic	Focus	Question No.
Representing Meaning & Meaning Structure of Language	Analyze sentence representations, compare structural representations of language, and evaluate how different parsing approaches capture meaning and relationships among words.	Q1
Syntax-Driven Semantic Analysis	Analyze parsing strategies for incomplete and ambiguous inputs, evaluate parsing efficiency, and determine suitable methods for real-time language understanding.	Q2
Lexemes and Their Senses & Word Sense Disambiguation	Analyze ambiguity in natural language sentences, evaluate ambiguity resolution techniques, and determine the most effective approach for selecting intended meanings.	Q3
Representing Linguistically Relevant Concepts	Analyze grammatical constraints, agreement features, argument structures, and linguistic representations required for accurate language processing.	Q4

Information Retrieval & Syntax-Driven Semantic Analysis	Analyze large-scale parsing strategies, evaluate performance trade-offs, and justify efficient approaches for processing massive linguistic datasets.	Q5
---	---	----

Annexure A – Comparative Analysis

1. A language processing system must represent sentence structure for grammar checking and syntactic analysis. The developers are deciding between Context-Free Grammar (CFG) trees and dependency parsing structures. Analyze how these two approaches differ in representing sentence structure and justify which is more effective for capturing relationships between words.

2. A real-time application needs to parse user input efficiently, even when sentences are incomplete or ambiguous. The system currently uses top-down parsing, but an alternative is Earley parsing.

Analyze the differences between these parsing techniques and reason which is more suitable for such dynamic input conditions.

3. An NLP model must handle ambiguity in sentences such as “She saw the man with a telescope.” The designers are considering CFG, PCFG, and neural parsing techniques.

Analyze how these approaches differ in handling ambiguity and reason which method is most effective for real-world applications.

4. A grammar system needs to correctly handle subject–verb agreement and verb argument structures. The developers are comparing feature structures and subcategorization frames.

Analyze how these approaches differ and justify which provides better support for enforcing grammatical correctness.

5. A large-scale NLP system must perform fast and accurate dependency parsing on big datasets. The choice is between transition-based parsing and graph-based parsing.

Analyze the differences between these methods in terms of decision-making and performance, and justify which is more suitable for large-scale applications.

Rubrics for Calculation

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Scope of Items	20%	Selects highly relevant items with clear scope and justification	Selects appropriate items with minor gaps	Selection is limited or partially relevant	Items are unclear or not relevant
Comparison Depth	25%	Provides detailed and comprehensive comparison across	Provides adequate comparison with some depth	Limited comparison with few aspects	Very minimal or no comparison

		multiple aspects			
Use of Evidence	20%	Supports comparison with accurate data, examples, or references	Uses some supporting evidence with minor gaps	Limited or weak supporting evidence	No supporting evidence provided
Critical Thinking	20%	Demonstrates strong critical thinking with clear interpretation and reasoning	Shows reasonable interpretation with minor gaps	Basic interpretation ; lacks depth	No meaningful interpretation
Clarity of Presentation	15%	Well-organized, clear, and logically structured presentation	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Q. No. / Criteria Level	Scope of Items	Comparison Depth	Use of Evidence	Critical Thinking	Clarity of Presentation	Sub. Total	Total %
1							
2							
3							
4							
5							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

1. Context-Free Grammar (CFG) Trees vs Dependency Parsing

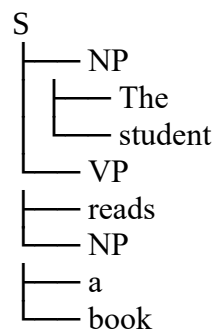
A language processing system can represent sentence structure using either Context-Free Grammar (CFG) trees or dependency parsing structures. Both methods describe grammatical relationships, but they represent those relationships differently.

CFG represents a sentence using a hierarchical phrase structure. A sentence is divided into constituents such as noun phrases (NP), verb phrases (VP), and prepositional phrases (PP).

Example:

Sentence: "The student reads a book."

CFG representation:



Here, CFG shows how individual words combine to form larger grammatical units.

Dependency parsing, on the other hand, represents direct relationships between individual words. One word acts as the head of another word.

For the same sentence, the dependency relationships can be represented as:

reads → student
reads → book
student → The
book → a

Here, "reads" is the main verb, "student" is the subject, and "book" is the object.

Comparison:

CFG:

- Represents hierarchical phrase structure.
- Groups words into phrases.
- Useful for syntactic and grammatical analysis.
- Clearly represents constituents.
- Can become complex for long-distance relationships.

Dependency Parsing:

- Represents direct word-to-word relationships.
- Identifies subjects, objects, modifiers, and other grammatical dependencies.
- Useful for information extraction and semantic analysis.
- Handles long-distance relationships more naturally.
- Produces a compact representation of sentence structure.

CFG is particularly useful when the main requirement is to understand the internal structure of phrases. For example, it can identify a noun phrase or verb phrase.

Dependency parsing is more effective when the objective is to understand relationships between words. For example, it can directly identify that "student" is the subject of "reads" and "book" is its object.

Therefore, dependency parsing is generally more effective for applications that require word-level grammatical relationships, such as information extraction, question answering, semantic analysis, and relation extraction. CFG remains useful for grammar checking and constituent-based syntactic analysis.

Conclusion:

CFG is better for representing hierarchical phrase structure, while dependency parsing is better for representing direct relationships between words. For modern NLP applications that focus on relationships between entities and actions, dependency parsing is generally more effective.

2. Top-Down Parsing vs Earley Parsing

Top-down parsing and Earley parsing are two techniques used to construct syntactic structures from input sentences. They differ mainly in how they search for valid parse structures.

Top-down parsing starts from the start symbol of the grammar, usually S, and attempts to generate the input sentence by repeatedly applying grammar rules.

For example:

$S \rightarrow NP VP$

The parser first identifies the noun phrase and then the verb phrase before matching them with the input.

Top-down parsing is simple and works well for small and unambiguous grammars. However, it may require backtracking when the selected grammar rule does not match the input.

The major limitations of top-down parsing are:

1. It may require considerable backtracking.
2. Ambiguous sentences can produce multiple possible parsing paths.
3. Repeated parsing of the same structures can increase computation.
4. It is less suitable for incomplete sentences.
5. It can become inefficient for large and complex grammars.
6. Real-time applications may experience delays when extensive backtracking is required.

Earley parsing is a chart parsing technique that can handle general Context-Free Grammars. Instead of repeatedly starting parsing from the beginning, it stores partial parsing states in a chart.

Earley parsing uses three main operations:

1. Prediction – predicts possible grammar rules.
2. Scanning – matches grammar symbols with input words.
3. Completion – completes partially recognized grammar structures.

For example, if a real-time system receives:

"Book a flight to..."

Earley parsing can maintain the partial parsing state and continue processing when the remaining input arrives.

Comparison:

Top-Down Parsing:

- Starts from the grammar start symbol.
- Simple to implement.

- Suitable for simple grammars.
- Can require backtracking.
- Less suitable for ambiguous input.
- Less suitable for incomplete input.
- Can become inefficient for complex grammars.

Earley Parsing:

- Uses chart-based parsing.
- Handles ambiguous grammars.
- Supports incomplete or partial input.
- Avoids repeated processing of the same partial structures.
- Handles general CFGs.
- More suitable for complex and dynamic input.
- Requires additional memory for the chart.

For a real-time application that receives incomplete, ambiguous, or continuously changing user input, Earley parsing is more suitable. It can maintain partial hypotheses and continue parsing as more input becomes available.

However, for very simple and predictable input, top-down parsing can be faster and easier to implement.

Conclusion:

Top-down parsing is appropriate for simple and controlled grammar environments, while Earley parsing is better for dynamic applications involving ambiguous, incomplete, and complex input. Therefore, Earley parsing is the preferred approach for real-time applications that need flexible and robust parsing.

3. CFG, PCFG, and Neural Parsing for Ambiguous Sentences

Consider the sentence:

"She saw the man with a telescope."

This sentence is structurally ambiguous because the phrase "with a telescope" can have two different interpretations.

Interpretation 1:

She used a telescope to see the man.

In this interpretation, "with a telescope" modifies the verb "saw".

The structure is approximately:

She → saw → man
|
with a telescope

Interpretation 2:

The man had a telescope.

In this interpretation, "with a telescope" modifies the noun "man".

The structure is approximately:

She → saw → man
|
telescope

Therefore, the same sentence can have multiple valid parse trees.

CFG handles this ambiguity by generating multiple possible parse structures. It can represent both interpretations, but a basic CFG does not provide a mechanism for deciding which interpretation is more likely.

For example, CFG may contain rules such as:

$S \rightarrow NP VP$
 $VP \rightarrow V NP$
 $VP \rightarrow VP PP$
 $NP \rightarrow NP PP$

These rules allow the prepositional phrase to attach to either the verb phrase or the noun phrase.

The major limitation is that CFG treats the possible structures primarily as grammatical alternatives. It does not inherently use probability or real-world context to select the intended meaning.

PCFG extends CFG by assigning probabilities to grammar rules.

For example:

VP → VP PP 0.70

NP → NP PP 0.30

If the first interpretation has a higher probability based on training data, the PCFG can select it as the preferred parse.

Therefore, PCFG is better than basic CFG for resolving structural ambiguity because it ranks alternative parse trees according to their probability.

Neural parsing uses machine-learning models to learn syntactic and contextual patterns from large amounts of language data. Neural models can use the surrounding words and contextual representations to determine which interpretation is more likely.

For example, a neural parser can learn that different words following "with" can influence the interpretation of the phrase.

Comparison:

CFG:

- Uses fixed grammar rules.
- Can generate multiple parse trees.
- Does not naturally rank alternatives.
- Requires fewer training resources.
- Suitable for controlled grammar applications.

PCFG:

- Extends CFG with probabilities.
- Can rank alternative parse trees.
- Uses corpus-based information.
- Better at resolving ambiguity than CFG.
- Still depends on grammar rules and probability estimates.

Neural Parsing:

- Learns patterns from data.
- Uses contextual information.
- Can handle complex linguistic ambiguity.
- Usually provides better performance on real-world language.
- Requires large training datasets and computational resources.

For real-world NLP applications, neural parsing is generally the most effective because language ambiguity depends heavily on context, semantics, and usage patterns.

However, PCFG can be useful when a system requires an interpretable probabilistic grammar. CFG is suitable when the language is restricted and predictable.

Conclusion:

CFG can represent ambiguity, PCFG can rank ambiguous structures using probabilities, and neural parsing can use contextual information learned from large datasets. Therefore, neural parsing is generally the most effective approach for real-world applications, while PCFG provides a useful intermediate solution between traditional grammar-based parsing and neural approaches.

4. Feature Structures vs Subcategorization Frames

Feature structures and subcategorization frames are both used to improve grammatical analysis, but they address different aspects of language.

Feature structures represent grammatical properties associated with words or phrases. Common features include:

- Person
- Number
- Gender
- Tense
- Case
- Agreement

For example:

"The student reads."

The subject "student" has:

Number = Singular

Person = Third

The verb "reads" also has:

Number = Singular

Person = Third

Tense = Present

The features are compatible, so the sentence satisfies subject-verb agreement.

Consider:

"The students reads."

Here:

Students = Plural

Reads = Singular

The feature structures detect the mismatch and identify a subject-verb agreement error.

Therefore, feature structures are particularly useful for enforcing grammatical constraints.

Subcategorization frames describe the types of arguments that a verb requires or allows.

For example:

"eat" can be represented as:

eat + NP

Example:

"The student eats food."

The verb "give" can take two objects:

give + NP + NP

Example:

"The teacher gave the student a book."

The verb "depend" can require a prepositional phrase:

depend + PP

Example:

"The result depends on the data."

Subcategorization therefore focuses mainly on verb argument structures.

Comparison:**Feature Structures:**

- Represent grammatical features.
- Handle subject-verb agreement.
- Represent number and person.
- Represent tense and case.
- Useful for checking grammatical compatibility.
- Focus on grammatical constraints.

Subcategorization Frames:

- Represent verb argument requirements.
- Identify the number and type of arguments expected by a verb.
- Useful for semantic role analysis.
- Help identify appropriate objects and complements.
- Focus on verb-argument relationships.

For example:

"The doctor prescribed medicine to the patient."

The verb "prescribed" can have a subcategorization structure such as:

prescribe + NP + NP

The system can identify:

Agent = Doctor

Theme = Medicine

Recipient = Patient

Feature structures alone would not fully represent these verb-argument relationships.

On the other hand, subcategorization frames alone would not be sufficient for enforcing subject-verb agreement.

Therefore, if the main requirement is grammatical correctness and subject-verb agreement, feature structures provide better support.

If the main requirement is identifying the arguments required by verbs, subcategorization frames are more suitable.

The best solution is to combine both approaches:

Feature Structures + Subcategorization Frames → Grammatical and Syntactic Validation

Conclusion:

Feature structures are better for enforcing grammatical correctness, especially subject-verb agreement, while subcategorization frames are better for representing verb argument structures. A combined system provides the most complete grammatical analysis.

5. Transition-Based Parsing vs Graph-Based Parsing

Transition-based and graph-based dependency parsing are two major approaches for constructing dependency trees.

Transition-based parsing constructs a dependency tree step by step using a sequence of parsing actions.

Common actions include:

SHIFT
REDUCE
LEFT-ARC
RIGHT-ARC

For example, the parser may process a sentence as follows:

Input → SHIFT → LEFT-ARC → RIGHT-ARC → Dependency Tree

At each stage, the parser makes a decision about the next transition. Modern transition-based parsers often use machine-learning or neural models to select the best transition.

Advantages of transition-based parsing include:

1. Very fast processing.
2. Low memory requirements.
3. Suitable for real-time applications.
4. Efficient for large datasets.
5. Can process sentences with approximately linear complexity in many practical systems.

A major disadvantage is that early mistakes can affect later parsing decisions. This is known as error propagation.

Graph-based parsing treats dependency parsing as a global optimization problem.

Instead of constructing the tree one transition at a time, the parser considers possible dependency relationships between words and assigns scores to them.

It then searches for the highest-scoring complete dependency tree.

For example:

```
      reads
     /   \
student   book
```

The parser evaluates possible relationships and selects the dependency tree with the best overall score.

Advantages of graph-based parsing include:

1. Considers global sentence structure.
2. Better at handling long-distance dependencies.
3. Less dependent on earlier local decisions.
4. Can produce globally optimized dependency trees.
5. Often provides strong parsing accuracy.

However, graph-based parsing is generally more computationally expensive and requires more memory than transition-based parsing.

Comparison:**Transition-Based Parsing:**

- Makes local decisions.
- Builds the tree step by step.
- Very fast.
- Requires less memory.
- Suitable for real-time applications.
- Suitable for large-scale datasets.
- Can suffer from error propagation.
- Long-distance dependencies can be more difficult.

Graph-Based Parsing:

- Considers the entire dependency structure.
- Searches for a globally optimal tree.
- More computationally expensive.
- Requires more memory.
- Better at global structural relationships.
- Handles long-distance dependencies well.
- Generally slower than transition-based parsing.

For a large-scale NLP system processing millions of sentences, speed and scalability are extremely important. Transition-based parsing is therefore generally more suitable because it provides fast parsing with relatively low computational requirements.

For applications where maximum structural accuracy is more important than processing speed, graph-based parsing may be preferable.

For example:

Search engines, social media analysis, voice assistants, and real-time monitoring systems can benefit from transition-based parsing because they must process large volumes of text quickly.

Applications requiring detailed linguistic analysis can use graph-based parsing when additional computational cost is acceptable.

Conclusion:

Transition-based parsing is generally more suitable for large-scale and real-time NLP applications because of its speed, low memory requirements, and scalability. Graph-based parsing is preferable when global structural accuracy and long-distance dependency analysis are more important.